

第Ⅲ篇 Linux 与 PCI 总线

本篇主要讲述 Linux 系统与 PCI/PCIe 总线相关的一些内容，其重点在于 Linux 系统 PCI/PCIe 总线驱动程序的实现。并以此为基础说明 PCI 总线控制器及其相关设备在系统软件的初始化过程。本篇并不会拘泥于 Linux 系统的实现细节，但是仍将介绍一些与 Linux 系统相关的基本知识。本篇内容基于 Linux 2.6.31.6 内核。

值得注意的是，在不同处理器体系结构中，Linux 系统初始化 PCI 总线的过程并不相同。如在 Linux x86 系统中，BIOS 为 PCI 总线的初始化做出了许多辅助工作，而在 Linux PowerPC 或者 Linux ARM 中使用的 Firmware，如 U-Boot，并没有做类似的工作。

从系统软件的角度来看，PCI 总线与 PCIe 总线的初始化过程和资源分配较为类似，为节约篇幅，本篇将 PCI 和 PCIe 总线统称为 PCI 总线，并将 Linux 系统的 PCI 和 PCIe 子系统简称为 Linux PCI。

在第 12.3 节中讲述了一个最基本的、基于 PCI 总线的 Linux 设备驱动程序。这个 PCI 设备驱动程序使用了一些 Linux 系统提供的标准 API 和数据结构，例如使用 `pci_resource_start` 和 `pci_resource_len` 函数获得该设备 BAR 空间的基地址和长度，并在 `request_irq` 函数中使用 `pci_dev→irq` 参数注册该设备使用的中断服务例程。

该 PCI 设备（Capric 卡）在驱动程序中使用的这些存储器资源，由系统软件对 PCI 总线进行初始化时确定，而中断资源在使能相应的 PCI 设备时由系统软件分配。这个系统软件包括操作系统和 Firmware^①。

与其他处理器系统相比，x86 处理器作为一个通用处理器平台，始终强调向前兼容的重要性。而实现向前兼容需要做出许多牺牲，这也造成了 Linux x86 对 PCI 总线的初始化过程最为复杂也最为烦琐，x86 处理器在引入了 ACPI（Advanced Configuration and Power Interface）机制之后，方便了处理器系统对“不规范外部设备”的管理，但是使得 PCI 总线的初始化过程更为复杂。

下文将以 Linux x86 为主线说明 PCI 总线的初始化过程。Linux x86 在对 PCI 总线进行初始化之前，BIOS 对 PCI 总线做出了部分初始化工作，如创建 ACPI 表、预先分配 PCI 设备使用的存储器资源，并执行 PCI 设备 ROM 中的初始化代码等一系列步骤。

Linux x86 将继承 BIOS 对 PCI 总线的初始化成果，并在此基础上进行 Linux PCI 子系统

① x86 处理器使用的 BIOS 也是 Firmware 的一种。

的初始化，并执行 PCI 设备的 Linux 驱动程序的初始化模块。在 Linux x86 中，PCI 总线的初始化由一系列模块协调完成。

Linux x86 首先使用 “make menuconfig” 命令对内核进行必要的配置，然后产生 .config 文件。假定在 .config 文件中，CONFIG_PCI、CONFIG_PCI_MSI、CONFIG_PCI_GOANY、CONFIG_PCI_BIOS、CONFIG_PCI_DIRECT、CONFIG_PCI_MMCONFIG 等一些必要的参数为 “y”，即使能 PCI 总线驱动、使能 MSI 中断请求机制等，而且对 x86 处理器非常重要的 CONFIG_ACPI 参数也为 “y”。

在 Linux PCI 中，有两个常用的数据结构，分别为 pci_dev 和 pci_bus 结构。这两个数据结构的定义在 ./include/linux/pci.h 文件中。其中 pci_dev 结构描述 PCI 设备，包括这个 PCI 设备的配置寄存器信息，使用的中断资源，还有一些和 SR-IOV 相关的参数。而 pci_bus 结构描述 PCI 桥，包括这个 PCI 桥的配置寄存器信息和一些状态信息。该结构中 self 参数值得注意，pci_bus→self 指向一个 pci_dev 结构，该结构用于 PCI 桥的上游总线访问 PCI 桥，此时 PCI 桥被当作一个设备。